

**Міністерство освіти і науки України
Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки
Кафедра обчислювальної техніки**

Лабораторна робота 1

з дисципліни
«Бази даних»

на тему

« Збір вимог та розробка схеми ER »

Виконали:

студенти групи ІМ-55

Бабенко Ярослав Володимирович

Резніков Нікіта Сергійович

Номер в списку - 3, 24

Перевірив:

Хмельницький А. А.

Київ 2026

1. Короткий виклад вимог

Призначення системи: Розробка спеціалізованої онлайн-платформи для збереження, систематизації та обговорення поетичних творів. Платформа надає авторам інструмент для публікації творчості, а читачам — можливість формувати власні тематичні збірки та взаємодіяти з авторами через систему відгуків.

Дані для зберігання: Система повинна зберігати облікові записи користувачів, тексти авторських віршів, створені тематичні збірки, а також залишені відгуки та числові оцінки.

Бізнес-правила та завдання користувачів:

1. Унікальна ідентифікація користувачів та захист від профілів-клонів

- **Детальна бізнес-логіка:** Платформа повинна чітко розрізняти своїх авторів та читачів. Реєстрація кількох облікових записів на одну електронну скриньку або використання зайнятого нікнейму має бути суворо заборонена, щоб уникнути плутанини під час входу в систему та маніпуляцій з рейтингами.
- **Проблема, яка могла виникнути:** Якщо ми просто вкажемо, що поля email та username є обов'язковими для заповнення, система перевірятиме лише наявність тексту. Вона дозволить створити хоч сто профілів з однаковою поштою. Більше того, без додаткових уточнень система може сприймати User@mail.com та user@mail.com як дві різні адреси, що дозволить зловмисникам створювати акаунти-клони, просто змінюючи регістр літер.
- **Рішення на рівні концептуальної моделі:** У нашій ER-моделі ми визначили атрибути email та username як **Альтернативні ключі (Alternate Keys)**. Це означає, що хоч вони і не є головним порядковим номером (user_id), вони наділені такою ж силою — кожен запис у цих полях має бути абсолютно унікальним у межах усієї системи. Додатково у специфікації ми прописали, що домен цих атрибутів є реєстронезалежним. Це гарантує цілісність бази користувачів ще на етапі її проектування.

2. Захист від штучної накрутки рейтингів (Один вірш — один відгук)

- **Детальна бізнес-логіка:** Платформа формує чесні топи та рейтинги творів на основі оцінок читачів. Тому базове правило звучить так: один користувач має право залишити рівно один відгук на один конкретний вірш. Якщо користувач змінив думку, він повинен відредагувати свій старий відгук, а не створювати новий.

- **Проблема, яка могла виникнути:** Якби ми виділили «Відгук» як звичайну самотійну сутність і дали їй власний ідентифікатор (наприклад, `review_id`), модель би логічно дозволяла одній людині створити скільки завгодно відгуків до одного вірша (адже кожен новий відгук мав би свій унікальний `review_id`). Це прямий шлях до спаму та маніпуляції середнім балом твору.
- **Рішення на рівні концептуальної моделі:** Ми кардинально змінили підхід і зробили «Відгук» **Слабкою сутністю (Weak Entity)**. Ми повністю позбавили її власного ідентифікатора. Натомість вона отримала **Складений ідентифікатор (Composite Identifier)**, який утворюється з двох частин: ID користувача (хто пише) та ID вірша (що оцінюють). Структура такого ключа концептуально гарантує, що комбінація «Конкретний читач + Конкретний вірш» може існувати в системі лише в одному екземплярі.

3. Заборона самооцінювання та конфлікту інтересів

- **Детальна бізнес-логіка:** Автор не має права виступати рецензентом власних творів. Оцінювати можна виключно творчість інших користувачів платформи, інакше автори будуть постійно ставити собі найвищі бали для просування в топ.
- **Проблема, яка могла виникнути:** Графічна схема ER-діаграми показує два паралельні зв'язки: користувач може бути пов'язаний з віршем як «Автор» і як «Рецензент». Самі по собі лінії зв'язків на кресленні не здатні заборонити одному користувачеві одночасно виконувати обидві ролі для одного й того ж твору.
- **Рішення на рівні концептуальної моделі:** Оскільки графічно це не вирішити, ми доповнили модель строгим **Семантичним правилом (Semantic Constraint)** у супровідній документації. Це правило чітко вказує: екземпляр сутності «Користувач» не може вступати у зв'язок «Оцінює» з екземпляром сутності «Вірш», якщо між ними вже існує зв'язок «Є автором». Це концептуальне обмеження дає чітку вказівку розробникам реалізувати відповідні перевірки на етапі створення бази даних (наприклад, через тригери).

4. Підтримка повноцінного співавторства

- **Детальна бізнес-логіка:** Літературний твір не завжди створюється однією людиною. Часто це робота творчого тандему (наприклад, автор оригіналу та перекладач, або два поети-співавтори). Платформа повинна вміти закріплювати один вірш за кількома користувачами одночасно, зберігаючи їхні ролі.

- **Проблема, яка могла виникнути:** Якщо використовувати простий зв'язок «Один-до-багатьох» (1:N), вірш завжди матиме лише одного власника. Якщо ж спробувати записати кількох авторів в один атрибут, ми порушимо фундаментальне правило атомарності даних. А прямий зв'язок «Багато-до-багатьох» (M:N) концептуально не дозволяє вказати, яку саме роль виконував кожен із співавторів.
- **Рішення на рівні концептуальної моделі:** Ми виконали процес розв'язання зв'язку M:N. В модель була введена нова **Асоціативна сутність (Associative Entity)** `Poem_Author`. Вона розбила один складний зв'язок на два прості і зрозумілі. Головна перевага цього рішення — асоціативна сутність дозволила нам додати новий атрибут `contribution_type` (тип внеску). Тепер ми можемо не лише додати п'ятьох авторів до одного вірша, а й чітко вказати, хто з них головний автор, хто редактор, а хто перекладач.

5. Управління даними при видаленні профілю

- **Детальна бізнес-логіка:** Користувач має повне право видалити свій обліковий запис із платформи. При цьому система повинна автоматично прибрати всі створені ним приватні збірки та залишені відгуки, щоб уникнути ситуації, коли на сайті відображаються оцінки від неіснуючих людей.
- **Проблема, яка могла виникнути:** Якщо зв'язки між сутностями спроектувати як незалежні, після видалення профілю користувача його відгуки та збірки залишаться "висіти" у системі як сироти. Це порушить логічну цілісність моделі, адже відгук не може існувати сам по собі — його обов'язково мав хтось написати.
- **Рішення на рівні концептуальної моделі:** Під час проєктування ми застосували концепцію **Екзистенційної залежності (Existence Dependency)**. Для зв'язків між Користувачем і Відгуком, а також між Користувачем і Збіркою, ми встановили **Обов'язковий клас належності (Mandatory Participation)** з боку підлеглих сутностей. Це означає, що відгук або збірка фундаментально не мають права існувати без свого власника. Це правило концептуально гарантує, що при видаленні головної сутності всі залежні від неї дані також будуть безслідно знищені.

6. Обов'язковість та межі оцінок (Контроль якості рейтингів)

- **Детальна бізнес-логіка:** Платформа математично вираховує середній рейтинг кожного твору. Щоб ця математика працювала правильно, оцінка є обов'язковою для кожного відгуку і повинна суворо відповідати п'ятибальній шкалі.

- **Проблема, яка могла виникнути:** Якби ми просто визначили атрибут «Оцінка» як числовий, система дозволила б зберегти відгук із балом -5, 0 або 1000. Потрапляння хоча б однієї такої аномальної оцінки в базу миттєво зламало б алгоритм розрахунку середнього бала, і рейтинг твору став би неадекватним.
- **Рішення на рівні концептуальної моделі:** Для атрибута rating ми жорстко прописали **Домен атрибута (Attribute Domain)**. Домен — це дозволений набір значень. У нашому випадку він обмежений лише множиною цілих чисел {1, 2, 3, 4, 5}. Крім того, ми визначили цей атрибут як **Обов'язковий (Mandatory)**, щоб користувач не міг залишити коментар, забувши поставити оцінку.

7. Управління станами контенту (Чернетки та публікація)

- **Детальна бізнес-логіка:** Написання вірша — це процес. Автор може тижнями тримати твір у чернетках, перш ніж показати його читачам. Відповідно, стрічка новин має сортуватися за часом фактичної публікації, а не за датою створення чернетки.
- **Проблема, яка могла виникнути:** Якби у вірша була лише одна дата («Дата створення»), старий твір, який автор писав пів року, після публікації опинився б у самому низу стрічки новин (бо він формально "старий"). До того ж, без системи статусів недописані тексти могли б випадково потрапляти в загальний доступ.
- **Рішення на рівні концептуальної моделі:** Ми додали до сутності «Вірш» **Атрибут стану (State Attribute) status** (який може бути тільки «чернеткою» або «опублікованим») та окремий часовий атрибут `published_at` (час публікації). У супровідній документації ми прописали правило: час публікації фіксується тоді й тільки тоді, коли статус змінюється на «опубліковано». Це вирішує всі проблеми з хронологією.

8. Захист від дублікатів у збірках (Чистота колекцій)

- **Детальна бізнес-логіка:** Будь-яка тематична збірка повинна містити унікальний набір творів. Немає жодного сенсу додавати один і той самий вірш до однієї збірки п'ять разів — це штучне роздування обсягу.
- **Проблема, яка могла виникнути:** Якщо просто сказати, що Збірка і Вірш пов'язані, ніщо не заважає системі створити цей зв'язок багато разів поспіль (наприклад, через випадковий подвійний клік користувача). Тоді статистика показуватиме, що у збірці 100 творів, хоча це буде 100 копій одного вірша.

- **Рішення на рівні концептуальної моделі:** Для нашої проміжної асоціативної сутності `Collection_Poem` ми створили **Складений головний ідентифікатор (Composite Primary Identifier)**. Він утворюється з ID збірки та ID вірша. Завдяки цьому пара «конкретна збірка + конкретний вірш» стає абсолютно унікальною. Спроба створити другу таку ж асоціацію концептуально відхиляється.

9. Хронологічний порядок у збірках (Пам'ять зв'язків)

- **Детальна бізнес-логіка:** Збірка — це своєрідний плейлист. Читачі очікують бачити вірші всередині збірки в тому порядку, в якому укладач їх туди додавав (від найновіших до найстаріших).
- **Проблема, яка могла виникнути:** Зв'язок «Багато-до-багатьох» констатує лише сам факт того, що вірш є у збірці. Він нічого не знає про час. Без додаткових атрибутів система виводила б вірші у хаотичному порядку, руйнуючи логіку впорядкування.
- **Рішення на рівні концептуальної моделі:** Ми наділили зв'язок власною "пам'яттю". До асоціативної сутності `Collection_Poem` ми додали **Атрибут зв'язку (Relationship Attribute) `added_at`**. Цей атрибут не стосується ні самого вірша, ні самої збірки — він описує *момент виникнення зв'язку між ними*. Це дозволить програмістам у майбутньому легко сортувати вірші за цим часовим маркером.

10. Заборона "невидимих" віршів (Правила змістовності)

- **Детальна бізнес-логіка:** Кожен твір на платформі повинен мати реальну назву та осмислений текст. Публікація абсолютно порожніх сторінок заборонена.
- **Проблема, яка могла виникнути:** Якщо зробити атрибути «Назва» та «Текст» просто обов'язковими (Mandatory), система не пропустить повну відсутність даних (Null), але з радістю збереже рядок, що складається лише з десяти пробілів чи переносів каретки. На сайті з'являться порожні блоки, на які неможливо натиснути, що зламає інтерфейс.
- **Рішення на рівні концептуальної моделі:** Ми прописали суворі **Правила якості даних (Data Quality Rules)**. Ми звузили домен для атрибутів `title` та `content`: вони повинні містити не просто будь-які символи, а обов'язкові значущі (алфавітно-цифрові) символи. Простими словами: модель вимагає, щоб у вірші була хоча б одна справжня літера чи цифра, унеможливлючи створення смітєвого контенту.

2. Список сутностей з їхніми атрибутами

Сильні (незалежні) сутності

1. Сутність: Користувач (User)

- **user_id** — Головний ідентифікатор (Primary Identifier). Однозначно визначає екземпляр користувача.
- **username** — Альтернативний ключ (Alternate Key), обов'язковий. Регістронезалежний псевдонім (забезпечує унікальність профілю).
- **email** — Альтернативний ключ (Alternate Key), обов'язковий. Регістронезалежна електронна адреса.
- **password_hash** — Обов'язковий атрибут. Зашифрований пароль.
- **registered_at** — Обов'язковий темпоральний атрибут. Фіксує час створення профілю.

2. Сутність: Вірш (Poem)

- **poem_id** — Головний ідентифікатор (Primary Identifier).
- **title** — Обов'язковий атрибут. Домен: рядок, що містить значущі символи (заборона повністю порожніх назв).
- **content** — Обов'язковий атрибут. Домен: текст із мінімальною допустимою довжиною символів.
- **status** — Обов'язковий атрибут стану. Домен: { 'чернетка', 'опубліковано' }.
- **published_at** — Умовний (опціональний) темпоральний атрибут. Заповнюється виключно при переході status у стан опубліковано.

3. Сутність: Збірка (Collection)

- **collection_id** — Головний ідентифікатор (Primary Identifier).
- **title** — Обов'язковий атрибут. Назва колекції.
- **visibility** — Обов'язковий атрибут стану. Домен: { 'приватна', 'публічна' }.
- **created_at** — Обов'язковий темпоральний атрибут. Дата створення збірки.

Слабкі та асоціативні сутності

4. Сутність: Відгук (Review) — Слабка сутність

- **Ідентифікатор:** Складений (Composite Identifier), формується через ідентифікуючі зв'язки від сутностей User та Poem. Власного унікального ідентифікатора не має.
- **rating** — Обов'язковий атрибут. Доменне обмеження: цілі числа від 1 до 5.
- **comment** — Опціональний (необов'язковий) атрибут. Текстова рецензія читача.
- **updated_at** — Обов'язковий темпоральний атрибут. Час останньої зміни або редагування відгуку.

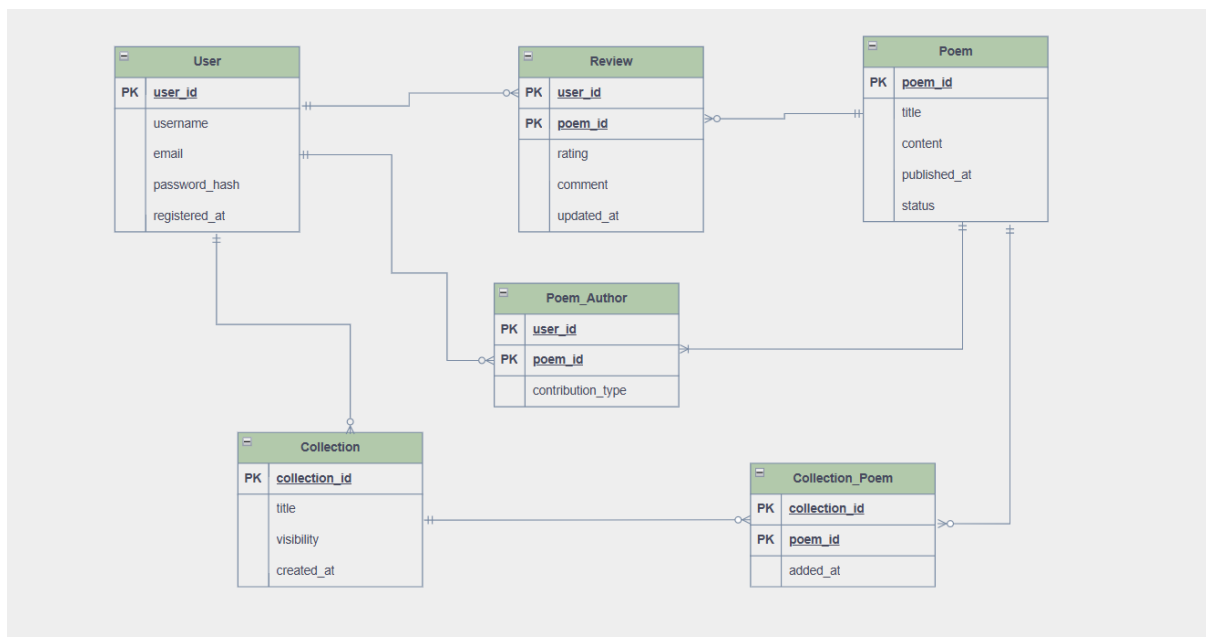
5. Сутність: Авторство Вірша (Poem_Author) — Асоціативна сутність

- **Ідентифікатор:** Складений (Composite Identifier) із батьківських сутностей User та Poem.
- **contribution_type** — Обов'язковий атрибут. Роль конкретного користувача у створенні твору (наприклад: головний автор, перекладач, редактор).

6. Сутність: Вміст Збірки (Collection_Poem) — Асоціативна сутність

- **Ідентифікатор:** Складений (Composite Identifier) із батьківських сутностей Collection та Poem.
- **added_at** — Обов'язковий атрибут зв'язку. Фіксує точний хронологічний момент додавання конкретного вірша до конкретної збірки.

3. Діаграма ER (Entity Relationship Diagram)



4. Пояснення зв'язків між сутностями

- **User — Poem (M:N):** Один користувач може бути автором багатьох віршів, а один вірш може створюватися кількома співавторами. Для розв'язання цього зв'язку «багато до багатьох» спроектовано асоціативну сутність Poem_Author, яка розбиває його на два зв'язки «один до багатьох» (1:N від User та 1:N від Poem).
- **User — Collection (1:N):** Один користувач може створити багато тематичних збірок, проте кожна збірка має лише одного творця.
- **User — Review (1:N):** Один користувач може залишити багато відгуків на різні твори, кожен відгук ідентифікує одного автора коментаря.
- **Poem — Review (1:N):** Один вірш може отримати велику кількість відгуків від різних читачів, але кожен запис відгуку пов'язаний виключно з одним твором.
- **Poem — Collection (M:N):** Один вірш може бути доданий до багатьох різних збірок, а кожна збірка може містити багато віршів. Для розв'язання цього зв'язку «багато до багатьох» сплановано асоціативну сутність Collection_Poem, яка розбиває його на два зв'язки «один до багатьох» (1:N від Collection та 1:N від Poem)